

VULNERABILITY ASSESSMENT REPORT

WebGoat Test Application

Prepared for: Future Interns Cyber Security Internship

Date: July 20, 2026

CIN ID: FIT/JUL26/CS9735

Prepared by:

Sharkjoe Andson Mwalukasa

DOCUMENT CONTROL

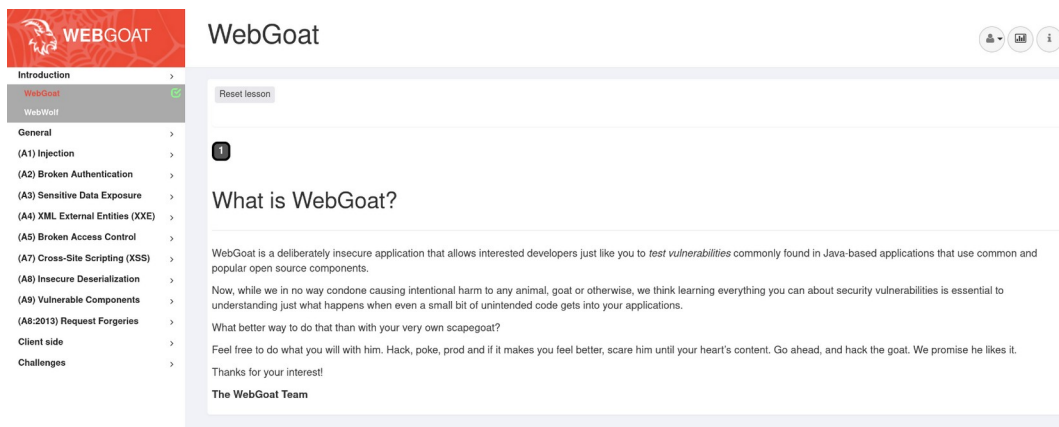
Document Title	Vulnerability Assessment Report WebGoat
Author	Sharkjoe Andson Mwalukasa
Organization	Future Interns
Classification	CONFIDENTIAL
Version	1.0

1. EXECUTIVE SUMMARY

This report presents the findings of a vulnerability assessment conducted on the WebGoat test application, a deliberately insecure web application maintained by the Open Web Application Security Project (OWASP). The assessment was performed as part of the Future Interns Cyber Security Internship program (CIN: FIT/JUL26/CS9735) between July 18, 2026, and July 20, 2026.

The primary objective of this assessment was to identify, classify, and document security vulnerabilities present in the target application using industry-standard tools and manual testing techniques. A total of twelve (12) security issues were identified, ranging from informational to high severity. Among these, two (2) vulnerabilities were classified as high severity, five (5) as medium severity, and five (5) as informational.

The most critical findings include SQL Injection and Spring4Shell (CVE-2022-22965), both of which could lead to unauthorized data access and remote code execution respectively. Additionally, multiple cookie security misconfigurations and the absence of anti-CSRF tokens present significant risks to session integrity and user authentication.



2. SCOPE AND METHODOLOGY

2.1 Scope

Target application	WebGoat v8.2.2
Target URL	http://localhost:8080/WebGoat
Target IP	127.0.0.1
Assessment type	White-box vulnerability assessment
Durationi	3 days (July 18 - July 20, 2026)

2.2 Tools used

Tools	Purpose	Version
Nmap	Network Scanning and port enumeration	7.99
OWASP ZAP	Automated vulnerabilities scanning	2.17.0
Mozilla Firefox	Manual Testing and DevTools inspection	Latest

2.3 Methodology

The assessment followed a structured approach comprising three phases:

Phase 1: Reconnaissance and Network Scanning

Network level reconnaissance was performed using Nmap to identify open ports, running services, and operating system information. A WHOIS lookup was also conducted to gather domain registration details.

```
(shark@mail)-[~]
$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.6.2 libssh2-1.11.1 libz-1.3.1 libpcre2-10.
46 libpcap-1.10.6 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select
```

```
$ whois webgoat.com
Domain Name: WEBGOAT.COM
Registry Domain ID: 80986420_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.godaddy.com
Registrar URL: http://www.godaddy.com
Updated Date: 2025-12-11T12:34:45Z
Creation Date: 2001-12-10T23:17:55Z
Registry Expiry Date: 2026-12-10T23:17:55Z
Registrar: GoDaddy.com, LLC
Registrar IANA ID: 146
Registrar Abuse Contact Email: abuse@godaddy.com
Registrar Abuse Contact Phone: 480-624-2505
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientRenewProhibited https://icann.org/epp#clientRenewProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Name Server: NS41.DOMAINCONTROL.COM
Name Server: NS42.DOMAINCONTROL.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-07-20T07:59:34Z <<<
```

```

(shark@mail)-[~]
$ nmap webgoat.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-20 10:57 +0300
Nmap scan report for webgoat.com (3.33.130.190)
Host is up (0.054s latency).
Other addresses for webgoat.com (not scanned): 15.197.148.33
rDNS record for 3.33.130.190: a2aa9ff50de748dbe.awsglobalaccelerator.com
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https

Nmap done: 1 IP address (1 host up) scanned in 10.88 seconds

```

```

(shark@mail)-[~]
$ nmap -sV webgoat.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-20 11:02 +0300
Nmap scan report for webgoat.org (188.114.97.3)
Host is up (0.042s latency).
Other addresses for webgoat.org (not scanned): 2a06:98c1:3121::3 2a06:98c1:3120::3 188.114.96.3
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Cloudflare http proxy
443/tcp   open  ssl/http  Cloudflare http proxy
8080/tcp  open  http      Cloudflare http proxy
8443/tcp  open  ssl/http  Cloudflare http proxy

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.66 seconds

```

```

L-$ sudo nmap -p- -sV -sC webgoat.org
[sudo] password for shark:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-20 11:05 +0300
Nmap scan report for webgoat.org (188.114.97.3)
Host is up (0.051s latency).
Other addresses for webgoat.org (not scanned): 2a06:98c1:3121::3 2a06:98c1:3120::3 188.114.96.3
Not shown: 65522 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Cloudflare http proxy
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare
443/tcp   open  ssl/http  Cloudflare http proxy
|_ssl-date: TLS randomness does not represent time
|_http-server-header: cloudflare
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_ssl-cert: Subject: commonName=webgoat.org
|_ Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org
|_ Not valid before: 2026-06-10T12:43:16
|_ Not valid after: 2026-09-08T13:43:03
2052/tcp  open  http      Cloudflare http proxy
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare
2053/tcp  open  ssl/http  nginx
|_ssl-date: TLS randomness does not represent time

```

```

|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
2096/tcp open  ssl/http nginx
|_ssl-date: TLS randomness does not represent time
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_ssl-cert: Subject: commonName=webgoat.org
| Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org
| Not valid before: 2026-06-10T12:43:16
| Not valid after: 2026-09-08T13:43:03
|_http-server-header: cloudflare
8080/tcp open  http      Cloudflare http proxy
|_http-server-header: cloudflare
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
8443/tcp open  ssl/http Cloudflare http proxy
|_ssl-date: TLS randomness does not represent time
|_ssl-cert: Subject: commonName=webgoat.org
| Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org
| Not valid before: 2026-06-10T12:43:16
| Not valid after: 2026-09-08T13:43:03
|_http-server-header: cloudflare
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
8880/tcp open  http      Cloudflare http proxy
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare

|_ssl-cert: Subject: commonName=webgoat.org
| Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org
| Not valid before: 2026-06-10T12:43:16
| Not valid after: 2026-09-08T13:43:03
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare
2082/tcp open  http      Cloudflare http proxy
|_http-server-header: cloudflare
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
2083/tcp open  ssl/http nginx
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare
|_ssl-cert: Subject: commonName=webgoat.org
| Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org
| Not valid before: 2026-06-10T12:43:16
| Not valid after: 2026-09-08T13:43:03
|_ssl-date: TLS randomness does not represent time
2086/tcp open  http      Cloudflare http proxy
|_http-title: Did not follow redirect to https://owasp.org/www-project-webgoat/
|_http-server-header: cloudflare
2087/tcp open  ssl/http nginx
|_ssl-date: TLS randomness does not represent time
|_ssl-cert: Subject: commonName=webgoat.org
| Subject Alternative Name: DNS:webgoat.org, DNS:*.webgoat.org

```

Phase 2: Automated Vulnerability Scanning

OWASP ZAP was configured to perform an automated spider and active scan against the target application. The scan was configured to use the traditional spider with Firefox as the browser engine. All discovered alerts were documented and categorized by severity.

The screenshot displays the OWASP ZAP Automated Scan interface. The top section, titled "Automated Scan", provides instructions and configuration options. The "URL to attack" is set to "http://localhost:8080/WebGoat". The "Use traditional spider" checkbox is checked, and the "Use ajax spider" dropdown is set to "If Modern" with "Firefox" as the engine. The "Attack" button is highlighted. Below this, a progress bar shows "37%" completion. The bottom section, "Active Scan", displays a table of results.

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. E
1,143	7/20/26, 11:34:19 AM	7/20/26, 11:34:19 AM	GET	http://localhost:8080/WebGoat/login?error=%3C...	200	OK	14 ms	255 bytes	1,810 bytes
1,144	7/20/26, 11:34:20 AM	7/20/26, 11:34:20 AM	POST	http://localhost:8080/WebGoat/login	200	OK	131 ms	255 bytes	1,888 bytes
1,145	7/20/26, 11:34:20 AM	7/20/26, 11:34:21 AM	POST	http://localhost:8080/WebGoat/register.mvc	200	OK	156 ms	255 bytes	5,020 bytes
1,146	7/20/26, 11:34:21 AM	7/20/26, 11:34:21 AM	GET	http://localhost:8080/WebGoat/login?error=%3C...	200	OK	16 ms	255 bytes	1,810 bytes
1,147	7/20/26, 11:34:21 AM	7/20/26, 11:34:21 AM	GET	http://localhost:8080/WebGoat/registration?nam...	200	OK	29 ms	255 bytes	4,605 bytes
1,148	7/20/26, 11:34:22 AM	7/20/26, 11:34:22 AM	GET	http://localhost:8080/WebGoat/start.mvc	200	OK	4 ms	255 bytes	1,810 bytes
1,149	7/20/26, 11:34:23 AM	7/20/26, 11:34:23 AM	GET	http://localhost:8080/WebGoat/registration?nam...	200	OK	31 ms	255 bytes	4,605 bytes

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites

Contexts

Default Context

Sites

Quick Start Request Response Requester

Header: Text Body: Text

HTTP/1.1 302 Found
Connection: keep-alive
Set-Cookie: JSESSIONID-HjEf_bkwtAYo0B2Hu0eXoL0oIvKfaps4GTRIL7; path=/WebGoat
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Location: http://localhost:8080/WebGoat/Login
Content-Length: 0

History Search Alerts Output Spider Active Scan

Alerts (11)

- SQL Injection
- Spring4Shell
- Absence of Anti-CSRF
- Content Security Policy
- Parameter Tampering
- Cookie No HttpOnly Flag
- Cookie without SameSite
- Authentication Request
- Session Management
- User Agent Fuzzer
- User Controllable HTML

Cookie No HttpOnly Flag

URL: http://localhost:8080/WebGoat/
Risk: Low
Confidence: Medium
Parameter: JSESSIONID
Attack:
Evidence: Set-Cookie: JSESSIONID
CWE ID: 1004
WASC ID: 13
Source: Passive (10010 - Cookie No HttpOnly Flag)
Input Vector:
Description: A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.
Other Info:

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites

Contexts

Default Context

Sites

Quick Start Request Response Requester

Header: Text Body: Text

HTTP/1.1 200 OK
Connection: keep-alive
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Content-Type: text/html; charset=UTF-8
Content-Language: en-US
Date: Mon, 20 Jul 2020 08:31:20 GMT
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Login Page</title>
<link rel="stylesheet" type="text/css" href="/WebGoat/css/main.css"/>
<link rel="stylesheet" type="text/css" href="/WebGoat/plugins/bootstrap/css/bootstrap.min.css"/>
<link rel="stylesheet" type="text/css" href="/WebGoat/css/font-awesome.min.css"/>

History Search Alerts Output Spider Active Scan

Alerts (11)

- SQL Injection
- Spring4Shell
- Absence of Anti-CSRF
- Content Security Policy
- Parameter Tampering
- Cookie No HttpOnly Flag
- Cookie without SameSite
- Authentication Request
- Session Management
- User Agent Fuzzer
- User Controllable HTML

User Controllable HTML Element Attribute (Potential XSS)

URL: http://localhost:8080/WebGoat/register.mvc
Risk: Informational
Confidence: Low
Parameter: matchingPassword
Attack:
Evidence:
CWE ID: 20
WASC ID: 20
Source: Passive (10031 - User Controllable HTML Element Attribute (Potential XSS))
Input Vector:
Description: This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
Other Info: User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL:

Phase 3: Manual Verification and Testing

Manual inspection was performed using browser developer tools to verify automated findings, inspect HTML source code, test for DOM-based Cross-Site Scripting (XSS), and analyze cookie security attributes.

3. FINDINGS AND ANALYSIS

3.1 HIGH SEVERITY VULNERABILITIES

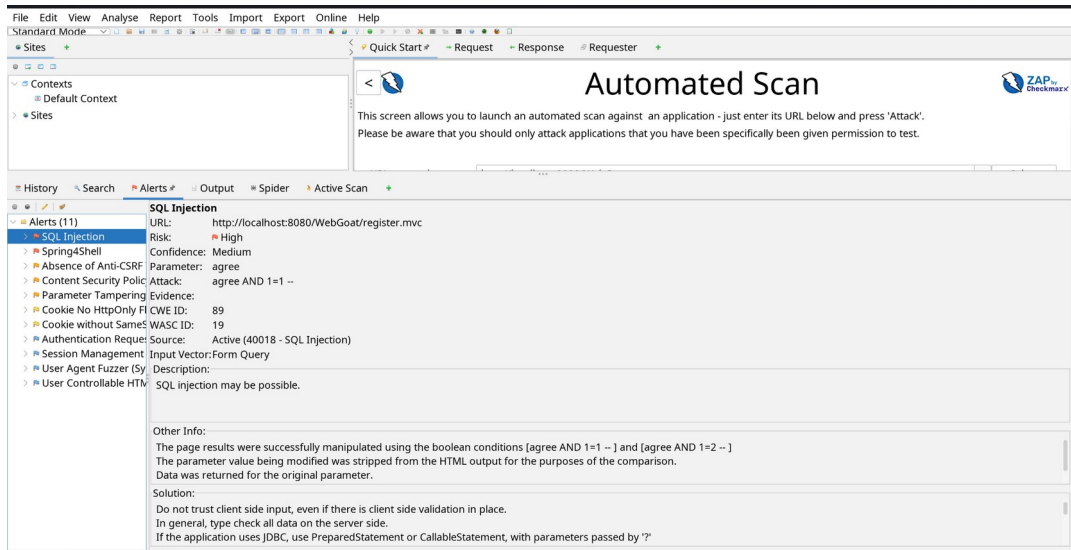
Vulnerability 1: SQL Injection

Severity	High
CWE ID	89
WASC ID	19
Url	http://localhost:8080/WebGoat/ register.mvc
Parameter	agree
Confidence	Medium

Description:

The application is vulnerable to SQL Injection through the 'agree' parameter in the registration form. The OWASP ZAP scanner successfully manipulated boolean conditions using the payloads 'agree AND 1=1 --' and 'agree AND 1=2 --', causing the application to return different responses based on the injected SQL conditions. This indicates that user-supplied input is being concatenated directly into SQL queries without proper sanitization or parameterization.

Impact: A successful SQL injection attack could allow an attacker to bypass authentication, extract sensitive database information, modify or delete data, and potentially execute administrative operations on the database server.



Remediation: Implement parameterized queries using PreparedStatement or CallableStatement in Java. All user-supplied input must be validated and sanitized on the server side. Employ the principle of least privilege for database accounts to minimize the impact of potential injection attacks.

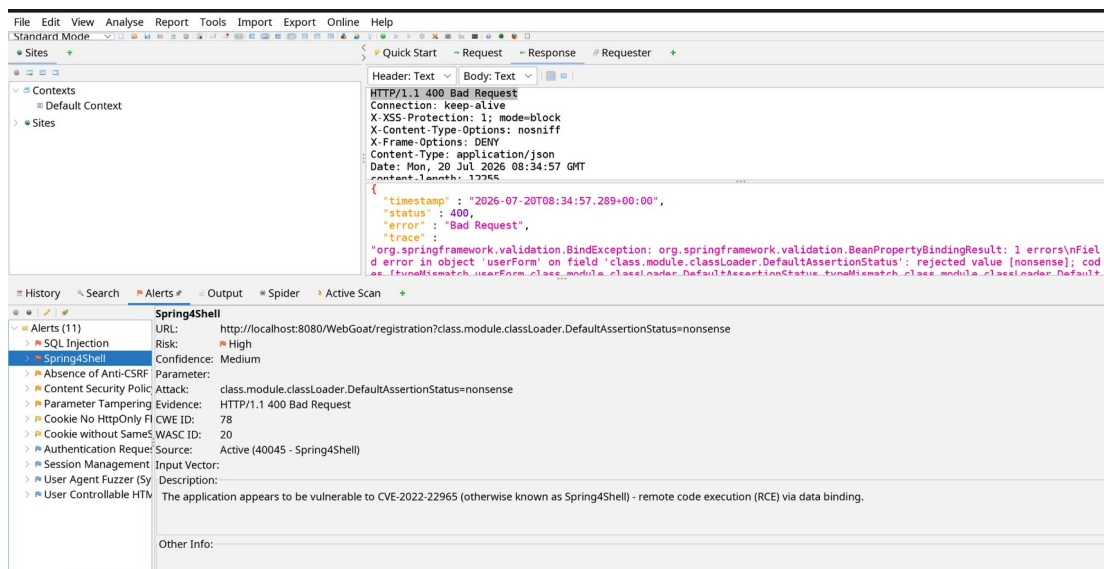
Vulnerability 2: Spring4Shell (CVE-2022-22965)

Severity	High
CWE ID	78
WASC ID	31
URL	http://localhost:8080/WebGoat/registration
Parameter	class.module.classLoarder.DefaultAssertionStatus
Confidence	Medium

Description:

The application is vulnerable to Spring4Shell (CVE-2022-22965), a critical remote code execution vulnerability affecting Spring Framework applications running on JDK 9+. The OWASP ZAP scanner detected that the application accepts the parameter 'class.module.classLoader.DefaultAssertionStatus' with a value of 'nonsense', which exploits the class loader manipulation mechanism in vulnerable Spring versions. This vulnerability allows attackers to write arbitrary files to the server, leading to remote code execution.

Impact: An attacker could exploit this vulnerability to execute arbitrary code on the server, gain complete control of the application, access sensitive data, pivot to internal networks, and deploy malware or ransomware.



Remediation: Immediately upgrade Spring Framework to version 5.3.18+ or 5.2.20+. If immediate patching is not possible, implement temporary mitigations by blocking requests containing 'class.module' or 'class.*' patterns at the web application firewall (WAF) level. Monitor for exploitation attempts in application logs.

3.2 MEDIUM SEVERITY VULNERABILITIES

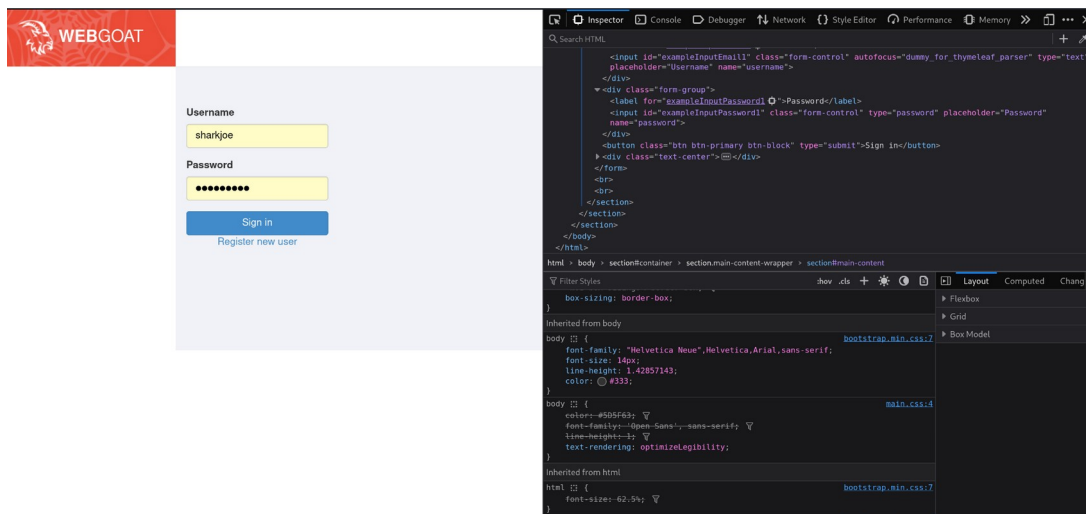
Vulnerability 3: Absence of Anti-CSRF Tokens

Severity	Medium
CWE ID	352
WASC ID	9
URL	http://localhost:8080/WebGoat

Description:

The application lacks Cross-Site Request Forgery (CSRF) protection tokens in its forms. Manual inspection of the login form HTML source code confirmed that there are no hidden input fields containing CSRF tokens. The form submits username and password data via POST method to /WebGoat/login without any mechanism to verify that the request originated from a legitimate user session.

Impact: An attacker could craft malicious web pages that submit unauthorized requests to the application on behalf of authenticated users. This could lead to unauthorized actions such as password changes, privilege escalation, or data manipulation.



Remediation: Implement synchronized token pattern by adding CSRF tokens to all state-changing forms. Use Spring Security's built-in CSRF protection or manually generate and validate tokens. Ensure tokens are cryptographically random, session-bound, and verified on every POST, PUT, DELETE, and PATCH request.

Vulnerability 4: Content Security Policy (CSP) Header Not Set

Severity	Medium
CWE ID	693
WASC ID	15
URL	http://localhost:8080/WebGoat

Description: The application does not implement a Content Security Policy (CSP) header. CSP is a critical security mechanism that helps prevent various attacks including Cross-Site Scripting (XSS), data injection, and clickjacking by specifying approved sources of content that browsers may load.

Impact: Without CSP, the application is more susceptible to XSS attacks, data injection, and other code execution vulnerabilities. Attackers can inject and execute malicious scripts from arbitrary sources.

The screenshot displays the Burp Suite interface. The top pane shows an HTTP response from http://localhost:8080/WebGoat. The response headers include: HTTP/1.1 200 OK, Connection: keep-alive, X-XSS-Protection: 1; mode=block, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Content-Type: text/html; charset=UTF-8, and Content-Language: en-US. The response body contains HTML for a login page, including a title and several CSS links. The bottom pane shows an alert titled 'Content Security Policy (CSP) Header Not Set' with a URL of http://localhost:8080/WebGoat. The alert details include a risk level of Medium, a confidence of High, and a parameter of 'Content Security Policy (CSP) Header Not Set'. The alert also lists evidence such as 'Parameter Tampering Evidence', 'Cookie No httpOnly', 'Cookie without SameSite', 'Authentication Required', 'Session Management', 'User Agent Fuzzer', and 'User Controllable HTTP'.

Remediation: Implement a strict CSP header such as: Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; Start with Content-Security-Policy-Report-Only to monitor violations before enforcing.

Vulnerability 5: Parameter Tampering

Severity	Medium
CWE ID	472
WASC ID	20
URL	http://localhost:8080/WebGoat/register.mvc

Description:

The OWASP ZAP scanner identified that the 'matchingPassword' parameter is susceptible to tampering. The application appears to rely on client-side validation for password confirmation, allowing attackers to manipulate the parameter value independently of the actual password field.

Impact: An attacker could bypass password confirmation checks, create accounts with mismatched passwords, or manipulate application logic by altering parameter values.

The screenshot displays the OWASP ZAP (Zed Attack Proxy) scanner interface. The top pane shows the 'Request' tab for an HTTP 1.1 500 Internal Server Error response. The bottom pane shows the 'Alerts' list, where a 'Parameter Tampering' alert is highlighted. The alert details include the URL 'http://localhost:8080/WebGoat/register.mvc', a 'Medium' risk level, and a 'Confidence: Medium' rating. The parameter being tampered with is 'matchingPassword'. The alert description states: 'Parameter manipulation caused an error page or Java stack trace to be displayed. This indicated lack of exception handling and potential areas for further exploit.'

Parameter Tampering

URL: http://localhost:8080/WebGoat/register.mvc

Risk: Medium

Confidence: Medium

Parameter: matchingPassword

Evidence: javax.servlet.http.HttpServlet.service(HttpServlet.java:517)\n\tat org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerAdapter.invokeHandlerMethod(AnnotationMethodHandlerAdapter.java:894)\n\tat org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerAdapter.handleInternal(AnnotationMethodHandlerAdapter.java:808)\n\tat org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerAdapter.handle(AnnotationMethodHandlerAdapter.java:87)\n\tat org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:1060)\n\tat org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:962)\n\tat org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1006)\n\tat org.springframework.web.servlet.FrameworkServlet.doPost(FrameworkServlet.java:909)\n\tat javax.servlet.http.HttpServlet.service(HttpServlet.java:517)\n\tat org.springframework.web.servlet.

Description: Parameter manipulation caused an error page or Java stack trace to be displayed. This indicated lack of exception handling and potential areas for further exploit.

Remediation: Implement server-side validation for all input parameters. Never trust client-side validation alone. Compare password and matchingPassword fields on the server before processing registration.

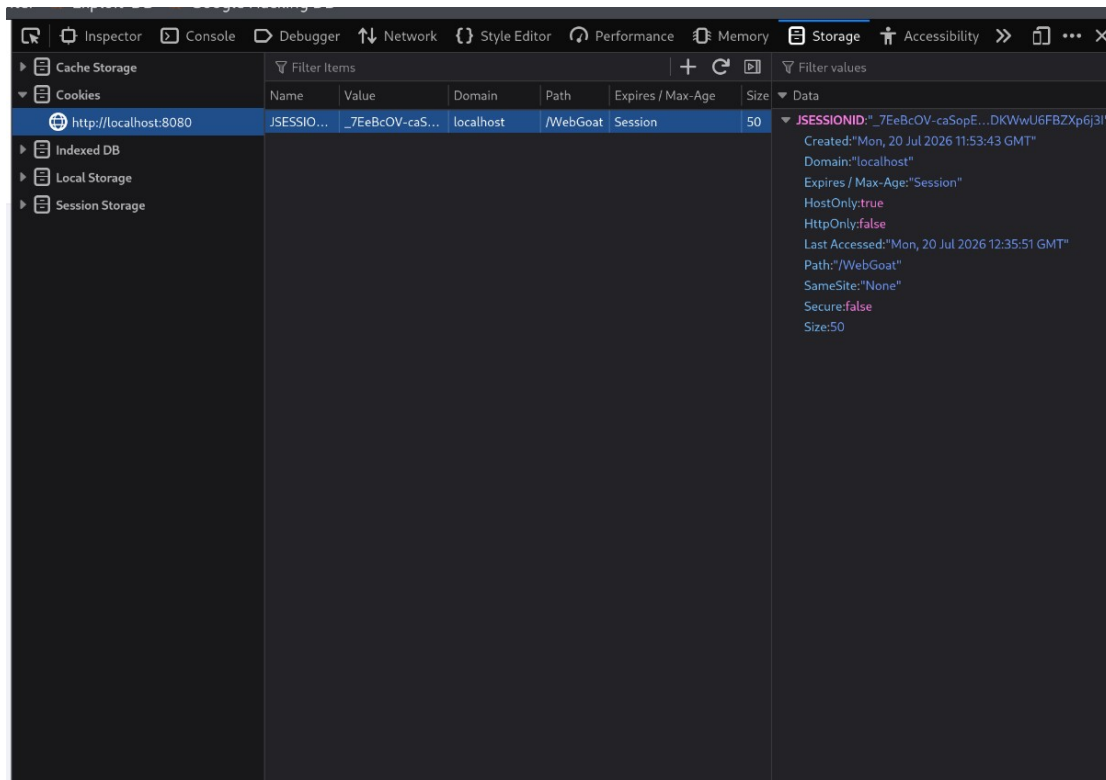
Vulnerability 6: Cookie No HttpOnly Flag

Severity	Medium
CWE ID	1004
WASC ID	13
URL	http://localhost:8080/WebGoat

Description:

The JSESSIONID cookie is set without the HttpOnly flag. Manual inspection of browser developer tools confirmed that the HttpOnly attribute is set to false. This allows client-side JavaScript to access the session cookie, making it vulnerable to theft through XSS attacks.

Impact: If an XSS vulnerability exists, an attacker could steal the session cookie and hijack the user's session, gaining unauthorized access to the application.



Remediation: Set the HttpOnly flag on all session cookies: `Cookie sessionCookie = new Cookie('JSESSIONID', sessionId); sessionCookie.setHttpOnly(true); sessionCookie.setSecure(true); sessionCookie.setSameSite(Cookie.SameSite.STRICT);`

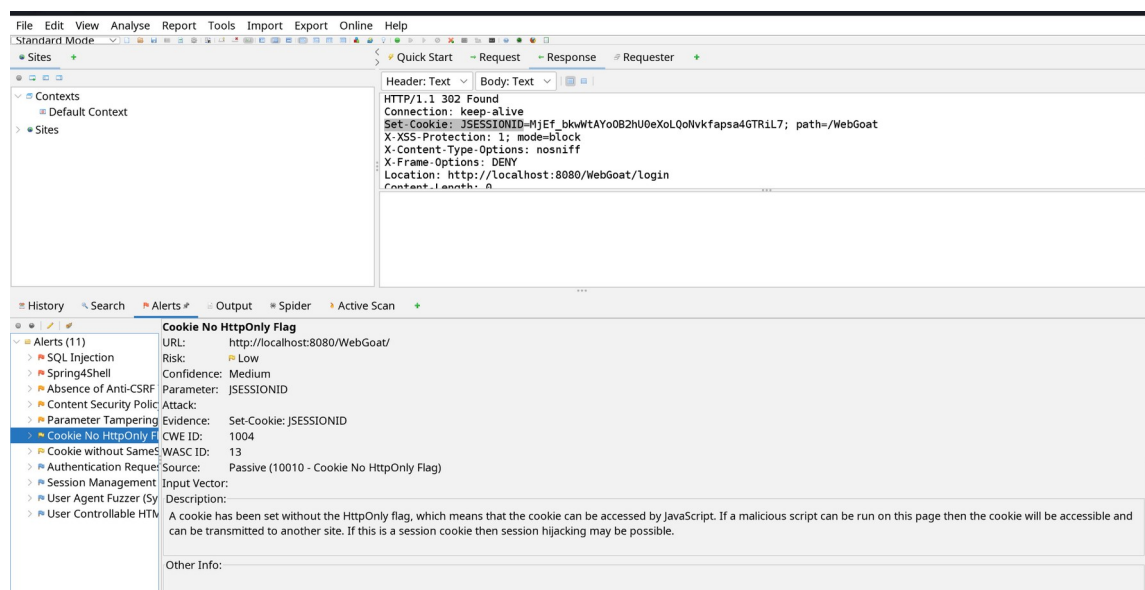
Vulnerability 7: Cookie Without SameSite Attribute

Severity	Medium
CWE ID	1275
WASC ID	13
URL	http://localhost:8080/WebGoat

Description:

The JSESSIONID cookie is set with SameSite: None, which allows the cookie to be sent with cross-site requests. This configuration exposes the application to Cross-Site Request Forgery (CSRF) attacks, as the browser will include the session cookie even when requests originate from different domains.

Impact: Attackers can exploit this to perform CSRF attacks, where malicious websites can make authenticated requests to the application using the victim's active session.



Remediation: Set SameSite=Strict or SameSite=Lax on all session cookies:
sessionCookie.setSameSite(Cookie.SameSite.STRICT);

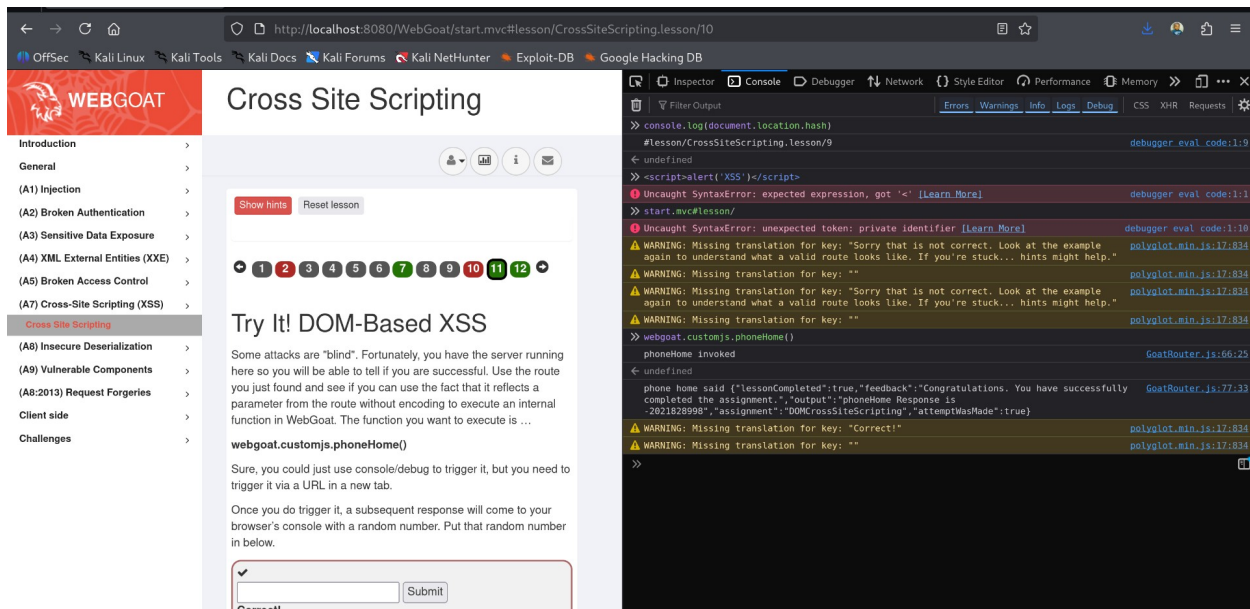
Vulnerability 8: DOM-Based Cross-Site Scripting (XSS)

Severity	Medium
CWE ID	79
WASC ID	8
URL	http://localhost:8080/WebGoat/start.mvc#lesson/CrossSiteScripting.lesson/9
Parameter	URL hash(DOM)

Description:

The application is vulnerable to DOM-Based XSS through improper handling of URL hash fragments. Manual testing confirmed that the application uses client-side JavaScript to process URL fragments (Backbone.js routing) without proper sanitization. The `phoneHome()` function could be invoked through DOM manipulation, demonstrating that attacker-controlled input reaches JavaScript execution contexts.

Impact: DOM-Based XSS allows attackers to execute malicious JavaScript in the victim's browser without server interaction. This can lead to session hijacking, credential theft, defacement, and malware distribution.



Remediation: Sanitize all DOM inputs using libraries like DOMPurify. Avoid using innerHTML, document.write(), or eval() with user-controlled data. Implement strict Content Security Policy to mitigate XSS impact.

3.3 INFORMATIONAL FINDINGS

#	Vulnerability	Notes
9	Authentication Request Identified	Login form detected normal behavior
10	Session Management Response	JSESSIONID cookie present normal behavior
11	User Agent Fuzzer	Different Response from different user agents
12	User Controllable HTML Element	matchingPassword field is user controlled

4. RISK SUMMARY MATRIX

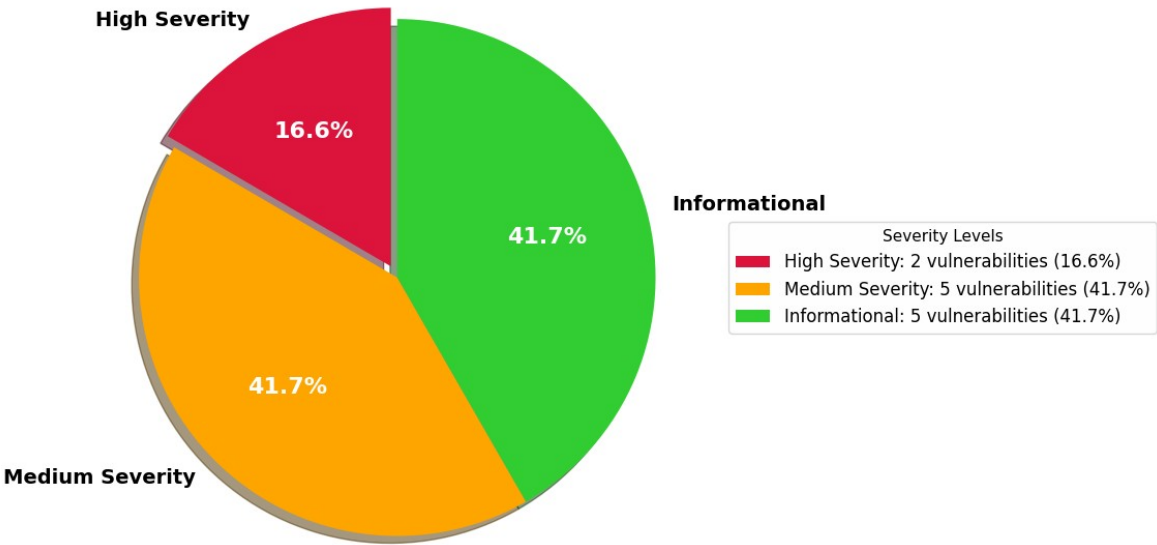
4.1 Vulnerability Distribution

Severity	Count	Percentage
High	2	16.6%
Medium	5	41.7%
Information	5	41.7%
Total	12	100%

4.2 Risk Classification by Category

Category	Vulnerabilities	Highest Severity
Injection	SQL Injection, Spring4Shell	High
Authentication	Missing CSRF, Cookie Issues	Medium
Client-side security	XSS, CSP Missing	Medium
Input Validation	Parameter Tampering	Medium
Session Management	Cookie Flags	Medium

ZAP Alerts Summary - Vulnerability Distribution
WebGoat Assessment (CIN: FIT/JUL26/CS9735)



5. REMEDIATION RECOMMENDATIONS

5.1 Immediate Actions (Within 7 Days)

Priority	Action	Vulnerability
P1	Patch Spring Framework to 5.3.18+	Spring4shell
P1	Implement parameterized queries	SQL Injection
P2	Add CSRF tokens to all forms	Absence of Anti-CSRF
P2	Set HttpOnly, Secure, SameSite flags	Cookie Security

5.2 Short-Term Actions (1-4 Weeks)

Priority	Action	Vulnerability
P3	Implement Content Security Policy	CSP missing
P3	Add Server-Side Input Validation	Parameter Tampering
P3	Sanitize DOM inputs with DOMPurify	DOM-Based XSS
P3	Implement output encoding	XSS Prevention

5.3 Long-Term Actions (1-3 Months)

Priority	Action	Purpose
P4	Conduct regular security assessment	Continuous improvement
P4	Implement Web Application Firewall (WAF)	Defense in depth
P4	Establish security coding standards	Prevention
P4	Implement security training for developers	Awareness

6. CONCLUSION

This vulnerability assessment of the WebGoat test application identified twelve (12) security issues, with two (2) classified as high severity. The findings demonstrate common web application vulnerabilities that organizations face in production environments.

The most critical issues — SQL Injection and Spring4Shell — require immediate attention due to their potential for complete system compromise. The medium-severity issues related to session management, input validation, and client-side security also present significant risks that should be addressed promptly.

All findings have been documented with clear evidence, impact assessments, and actionable remediation steps. Implementation of the recommended controls will significantly improve the security posture of the application.

This assessment was conducted in a controlled test environment using OWASP WebGoat, a deliberately vulnerable application designed for educational purposes. The skills and methodologies demonstrated here are directly applicable to real-world security assessments.

7. APPENDIX

Appendix A: OWASP ZAP Scan Configuration

Setting	Value
Target URL	http://localhost:8080/WebGoat
Spider type	Traditional
Browser	Firefox

Appendix B: Glossary

Term	Definition
CSRF	Cross Site Request Forgery
CSP	Content Security Policy
CWE	Common Weakness Enumeration
DOM	Document Object Model
HTTPOnly	Cookie flag preventing javascript access
SameSite	Cookie flag controlling cross site requests

SQLi	SQL Injection
WASC	Web application security consortium
XSS	Cross site scripting

END OF REPORT

